COMPUTABILITY / PROOF ENGINEERING

Shorter correctness proofs for K215

Path normalization, shared source trees, and a certified six-row machine transfer

Research contribution prepared for ProveIt
21 October 2026

Abstract

The primary 215-state consistency machine already has compiled Lean and Rocq endpoints. The useful next step is to reduce the additional proof required by a change, and to remove representational duplication from its existing certificate. We give two concrete contributions. First, identifying source paths that differ only in implicit trailing zero bits reduces the source certificate from 2,240 paired checkpoints to 1,426, a reduction of 814 checkpoints (36.34%). A new Lean checker proves the soundness of this quotient using the repository's existing segment-bisimulation theorem; the complete concrete replacement source theorem is kernel-checked. Second, the existing conditional-marker machine CM215 admits a short relative correctness proof: three local tape invariants imply that each K215 step is simulated by one or three CM215 steps. We provide a kernel-checked Lean proof for every initial binary tape and every initial head position, with initial control 0. A reusable positive-stuttering theorem is also checked in Coq. These results reuse the existing ZFC calibration.

The state-reduction search is reported separately and with its exact scope. This contribution does not claim a verified machine with 214 or fewer working states, or a lower bound of 215.

Repository snapshot: `VladimirReshetnikov/ProveIt`
Commit: `b6c7087477e51c017003bd729e712c8969733223`
Model: two symbols, one two-way infinite tape, blank start, L/R moves; the halt is excluded from the working-state count.

Contents

1 Answers and the exact verification boundary **2**

2 What has already been simplified **2**

3 Quotient source paths by their implicit zero extension **3**

3.1 The redundancy . . . 3

3.2 Why the quotient is sound . . . 4

3.3 Partition finite checks and keep shared trees shared . . . 5

4 CM215: a small relative correctness proof **6**

4.1 The exact six-row patch . . . 6

4.2 The three local invariants . . . 6

4.3 The one-step or three-step simulation . . . 7

4.4 Why positivity proves both directions . . . 7

5 Which 215-state alternative should be formalized next? **8**

6 The attempt to reach 214 states **9**

6.1 A finite invariant that does not depend on bounded blank runs . . . 9

6.2 A certificate format that transfers the existing endpoint . . . 10

6.3 Search family and measured result . . . 11

6.4 The next useful extension . . . 12

6.5 Why the obvious selector deletion is not free . . . 12

6.6 A better synthesis target . . . 12

7 Integration and reproducibility **13**

7.1 Suggested integration order . . . 13

7.2 Trusted and untrusted components . . . 13

8 Further research questions **13**

1 Answers and the exact verification boundary

How should the current proofs be shortened? Treat equivalent source addresses as the same checkpoint; preserve sharing when reflecting source trees into the kernel; and prove small machine changes directly against a previously verified machine. The first change reduces a specific current certificate by more than one third. The last change gives a small proof for an already published alternative without repeating the logical calibration, enumerator, compiler, runtime, and initialization developments.

Is another 215-state candidate easier to prove correct? Yes, in the important sense of *additional proof effort relative to the existing K215 endpoint*: the conditional-marker alternative CM215 is an especially good choice. It differs from K215 in six transition rows and has the same source program. Its relative proof works on arbitrary tapes. CM215 was already in the repository; the new contribution here is its compact checked relative proof and the proposed integration route, not a new discovery of the machine.

Is a correct 214-state candidate constructed here? No. The accompanying searches examine a precisely specified family of state-merger and local-macro transformations and find no accepted reduction. This is useful evidence about a search family, not a statement about all 214-state machines.

Item	Status and scope
Primary K215 consistency endpoint	Existing repository result in Lean and Rocq; the exact dense machine halts iff there is a finite ZFC refutation.
CM215 relative theorem	New Lean kernel check: arbitrary tape and head, initial control 0; exact literal tables; only standard logical axioms.
Positive-stuttering theorem	New standalone Lean and Coq proofs; no arithmetic or set-theoretic calibration assumptions.
Normalized source checker	New Lean soundness theorem; it saturates certificate entries under equality of zero-extended paths.
Concrete normalized certificate	Kernel-checked: the exact 1,426-entry certificate, both program/tree identities, and the unchanged all-register source-equivalence theorem. An independent finite reconstruction also passes.
214-state result	No verified candidate obtained; no minimality claim.

Table 1: The new work and its verification boundary.

This report fixes one repository snapshot. All machine bytes, source measurements, and new proofs refer to the commit on the title page. In particular, earlier descriptions of a 204-module Lean closure describe a historical fresh build; the current simplification record reports a 185-module closure with an incremental recheck. Neither number measures the size of the new relative proof [3, 4].

2 What has already been simplified

The primary endpoint has a deliberately layered structure [1, 2]. Its statement, in the repository's notation, is

```
BlankHalts machine215 <-> BProv ZFCax_s [] Form.fBot
```

and its nonhalting companion is obtained by negation. The endpoint does not prove unconditional nonhalting. The external theorem is also distinct from an object-language PA proof of the same equivalence; the latter remains a separate goal for this exact 215-state table.

Layer	Main responsibility
Logical calibration	Relate derivability of numerical falsity to finite ZFC refutation; handle arbitrary natural formula codes.
Proof enumeration	Show that the command language searches exactly the refutations relevant to the calibration.
Source correspondence	Relate the actual rotated source to the arithmetic/DAG enumerator.
Binary runtime	Interpret the source in the literal binary table, including registers, spares, decoder sites, and positive durations.
Startup and numbering	Connect the blank tape to the source start and the native labels to the published dense table.

Several attractive ideas are already implemented at the pin and should not be advertised as new recommendations. The forward source certificate already uses symmetry instead of a second mirrored certificate. The dominated implication proof already shares 61 bootstrap lemmas and 81 conditional schemas; its Lean source was reduced from 128,177 to 35,185 bytes. The enumerator theorem already handles general valid between-candidate starts. The site-coverage facade already reuses the same checked blocks with far less repeated path syntax. Recent commits also use symbolic command execution in `EnumPerm`, shared selector-level lemmas in `L2RootPerm`, and direct effects in the AX interpreter [4, 5].

Our changes address duplication that survives those improvements. They also suggest a policy for future work: maintain a small set of semantic interfaces, and make each candidate carry only the evidence that changes at one of those interfaces. The total state count and the extra proof cost are different objectives; a useful search should record both.

3 Quotient source paths by their implicit zero extension

3.1 The redundancy

The source decoder reads a finite Boolean path as an infinite stream padded with zeros. Thus the stored words P, $P0$, and $P00$ are different lists but designate the same stream. A certificate keyed by literal lists can store several checkpoints for the same semantic address.

Definition 3.1. For a finite Boolean word P, let $\mathrm{bit}(P, i)$ be its ith bit, or zero when i is outside the word. Define

$$P \sim Q \quad \Longleftrightarrow \quad \forall i \in \mathbb{N},\ \mathrm{bit}(P, i) = \mathrm{bit}(Q, i).$$

Let $\mathrm{can}(P)$ be P with all trailing zero bits removed.

Then $P \sim Q$ if and only if $\mathrm{can}(P) = \mathrm{can}(Q)$. Internal zeros and leading zeros must be retained. This is a semantic quotient of addresses, not a heuristic deletion of bits from a compiled program.

The pinned source certificate contains 2,240 paired checkpoints, all with offset zero. Normalizing both paths and preserving their paired successor links yields 1,426 entries: 1,425 decrement checkpoints and one halt checkpoint. Every decrement has two branch obligations, giving 2,850 branch segments

plus one initial segment, or 2,851 segment checks. The halt entry has its own terminal check. The 814 removed entries account for

$$\frac{2240 - 1426}{2240} = 0.363392857142\ldots.$$

These are counts of certificate obligations, not measured reductions in wall-clock time or in the entire proof development.

The certificate builder retains an explicit map from each old checkpoint to its new representative. It checks preservation of both successor links and both paths. Consequently, its numerical reduction can be audited without trusting a graph-minimization heuristic. More aggressive effect graph quotienting is possible, but is not needed to obtain almost all the observed sharing.

An independent reconstruction confirms that the quotient fibers consist of 707 singletons, 624 pairs, and 95 triples. Thus their number is 1,426 and their total population is $707 + 2 \cdot 624 + 3 \cdot 95 = 2240$. This second checker does not import the certificate producer's path or graph routines.

3.2 Why the quotient is sound

The repository already proves the critical semantic facts for its relation `RMEquiv.ipeq`: decoding and register execution respect equality of zero-extended streams. The new module proves the specific observation lemma needed by the existing certificate checker:

$$P \sim Q \implies \mathrm{tsite}(\mathrm{ptree}(p), P) = \mathrm{tsite}(\mathrm{ptree}(p), Q).$$

This transfers both the observed instruction and the consumed prefix. It is stronger than merely comparing the next opcode.

This distinction is essential: raw finite-path increment does not respect $\sim$. For example, $[1] \sim [1, 0]$, while their finite-path increments are $[0]$ and $[1, 1]$. The checker first obtains the equal *consumed leaf paths* from the decoder, and performs successor arithmetic on those paths. It does not increment a trimmed stored pointer directly.

Let C be the finite table of paired representatives. Instead of making the certificate relation literal table membership, define

$$\mathcal{R}_C(P, Q) \quad \iff \quad \exists (A, B) \in C, \quad P \sim A \ \wedge \ Q \sim B.$$

The relation is the saturation of the finite table under the semantic equivalence of paths. Each branch check compares its resulting paths with the stored successor representatives using a Boolean decision procedure for $\sim$, rather than ordinary list equality.

Theorem 3.2 (Normalized source-certificate soundness)**.** *Suppose the initial segment of each source reaches related checkpoints, and every entry of C passes the normalized checker. Then, for every register file R, the two programs halt alike from their empty-path configurations with registers R.*

Proof. The Boolean path comparator is proved sound for $\sim$. A successful branch check therefore supplies exactly the existing segment-bisimulation branch contract, with final paths related by $\mathcal{R}_C$. A successful entry check supplies matching halts or matching decrement instructions and their two branch contracts. For an arbitrary pair in the saturated relation, choose its stored representatives. The observation lemma makes the representative entry contract also a contract for the original pair.

Thus the finite certificate defines a bisimulation in the repository's existing sense. Its already proved halting-equivalence theorem gives the conclusion. No new induction over unbounded register values is required. □

The implementation is the theorem `check_halts_iff` in the module `RMNormalizedBisim`. Its axiom report is `[propext, Quot.sound]`; in particular, it does not assume consistency, formula well-formedness, or the truth of the numerical calculus.

3.3 Partition finite checks and keep shared trees shared

Two additional costs arose during verification. The pinned source data checks a 487-node well-formedness table in one kernel reduction, using linked-table lookups. It also prints reflected trees with repeated shared subtrees expanded at many occurrences. The unchanged data module failed to complete within the available memory during a fresh local attempt; isolating the components identified the monolithic well-formedness check as an immediate bottleneck. The failed attempts are environmental verification results, not evidence against the already published proof.

The first repair partitions that same finite well-formedness proposition into fifteen 32-element checks and one seven-element check. A small arithmetic argument assembles them into the original `wfCheck222` statement. This changes the reduction workload, not the program, the proposition, or the trusted axioms. The partitioned check compiled successfully in the new development.

The second repair is to name each distinct reflected subtree once, and prove its decoder correspondence from the correspondences of its children. A generic adequate-fuel statement avoids evaluating unnecessarily large fuel numerals at every occurrence. A small final theorem then identifies the shared reflected root with the exact program's tree. Sharing is safe because Lean checks those equalities; an untrusted generator merely chooses names and emits the finite DAG.

The implementation names 179 reachable source DAG nodes and 261 reachable target DAG nodes. Both reflected-root equalities are kernel-checked, together with the partitioned well-formedness proof. The completed replacement exports the original all-register source theorem under the same public name.

The compiled public facade retains the theorem `p215_halts_iff` in `KSource215`, with its original statement: for every register file R, p_{215} and `pathDagPerm` halt alike from the empty path with registers R. Its axiom report contains exactly `propext`, `Classical.choice`, and `Quot.sound`. This is a check of the complete source-correspondence replacement, separate from a fresh build of the entire machine-to-ZFC endpoint. The analogous normalized source checker in Rocq remains a porting task; the standalone Coq result in this packet is the positive-stuttering theorem below.

This optimization is independent of path normalization. Their combination addresses both the number of semantic obligations and the representation of the data used by each obligation. A reduction in generated source bytes must be reported separately from a reduction in kernel time, peak memory, or trusted assumptions.

4 CM215: a small relative correctness proof

4.1 The exact six-row patch

CM215 is the repository's conditional-marker alternative [6]. It retains 215 working states. In the following table all labels are native, and a row is written as *write, move, next control*.

Control	Read	K215	CM215
52	0	$0, R, 154$	$0, L, 176$
156	1	$1, R, 176$	$1, R, 52$
165	0	$0, R, 176$	$0, R, 52$
176	0	$0, R, 168$	$0, R, 185$
176	1	$1, R, 150$	$1, R, 129$
193	0	$0, R, 176$	$0, R, 52$

All other rows are unchanged. The two old decoder roles 52 and 176 agree on reading one, and differ on reading zero. Their origins can be recovered from the bit immediately to the left when the current bit is zero. CM215 uses control 52 for the two old roles and reuses physical label 176 as the selector. Define

$$\phi(q) = \begin{cases} 52, & q = 176, \\ q, & q \neq 176. \end{cases}$$

For a configuration $c = (q, h, T)$, let $\Phi(c) = (\phi(q), h, T)$. The map is not a bijective renaming; the interpretation of the reused physical label changes between the machines.

4.2 The three local invariants

For a K215 configuration (q, h, T), use the invariant

$$\begin{aligned} q = 52 &\implies T(h-1) = 1, \\ q = 176 \ \wedge \ T(h) = 0 &\implies T(h-1) = 0, \\ q = 156 &\implies T(h+1) = 1. \end{aligned}$$

It holds at initial control 0 for every tape and head. The third clause records the exceptional route through 156; retaining it is what makes the proof of preservation an ordinary one-step induction.

Lemma 4.1 (Local invariant preservation)**.** *Every K215 step preserves the three clauses above.*

Proof. The finite incoming-edge facts are checked against every row of the exact table. Each entry to 52 moves right after writing one, hence leaves one immediately to the left. Each entry to 156 moves left after writing one, hence leaves one immediately to the right. Each ordinary entry to 176 moves right after writing zero, leaving the required left zero. The sole exception is the read-one transition out of 156. By the third invariant, the cell to which it moves is one. Thus the antecedent $T(h) = 0$ of the second invariant is false on this exceptional entry. These facts cover all incoming rows, including those not observed on the blank run. □

The finite facts are kernel decisions over the literal table, and the tape argument is symbolic. There is no finite cutoff on tape length, register size, or elapsed run time.

4.3 The one-step or three-step simulation

Lemma 4.2 (Exact local simulation)**.** *For every K215 configuration satisfying the invariant, one K215 step is simulated from its image under Φ by either one or three CM215 steps. At the resulting boundary, control is related by ϕ, head positions agree, and the entire tape functions agree.*

Proof. Outside old controls 52 and 176, the row equality after applying ϕ is a finite check, including the three retargeted incoming rows. At either special control on reading one, the common transition gives a one-step simulation. At old control 52 on reading zero, CM215 moves left to its selector 176. The invariant supplies a one there. The selector moves right to provider 129, which realizes the old continuation to 154. The tape bits written on the detour equal the bits already present. At old control 176 on reading zero, the left marker is zero, so the same selector chooses provider 185 and realizes the continuation to 168. Both detours take three steps and leave the head and complete tape equal to the corresponding old one-step result. □

If $z(t)$ is the number of old steps among the first t using a read-zero row of 52 or 176, the constructed simulation schedule is exactly

$$\tau(t) = t + 2z(t), \qquad t \le \tau(t) \le 3t.$$

The inequalities express the positivity needed to reflect halting. They are not a state-count improvement.

4.4 Why positivity proves both directions

Let $f : A \to A$ and $g : B \to B$ be totalized deterministic step functions, and let H_A, H_B be terminal predicates. We totalize an actual halting configuration by a self-step; this convention does not add a working state or a legal stay-put transition to the released machine.

Theorem 4.3 (Positive-stuttering transfer)**.** *Suppose I is invariant under f, $I(a_0)$ holds, H_B persists under g, and $H_B(\Phi(a)) \leftrightarrow H_A(a)$. Suppose also that for every a satisfying I there is a natural number $k \ge 1$ with*

$$g^k(\Phi(a)) = \Phi(f(a)).$$

Then

$$(\exists n\ H_A(f^n(a_0))) \quad \Longleftrightarrow \quad (\exists m\ H_B(g^m(\Phi(a_0)))).$$

No uniform upper bound on the positive macro lengths is needed.

Proof. Induction constructs, for every n, a time $t_n \ge n$ with $g^{t_n}(\Phi(a_0)) = \Phi(f^n(a_0))$. A source halt therefore yields a target halt at the corresponding boundary. Conversely, suppose the target is terminal at time m. Choose the boundary $t_m \ge m$. Persistence makes the target terminal at that boundary as well, and terminal reflection makes $f^m(a_0)$ terminal. Thus intermediate target steps cannot conceal an early target halt. The lower bound $t_n \ge n$ is precisely where positive, rather than merely nonnegative, macro lengths are used. □

This theorem is short in both assistants. The Coq version additionally exposes the useful conclusion $H_B(g^n(\Phi(a_0))) \Rightarrow H_A(f^n(a_0))$ under the same assumptions. Its assumption reports are closed under the global context, and recursive `coqchk` is included in the verification record.

Theorem 4.4 (Checked CM215 relative endpoint)**.** *For every binary tape $T : \mathbb{Z} \to \{0,1\}$ and every head position $h \in \mathbb{Z}$,*

$$\mathrm{Halt}(K_{215}, (0, h, T)) \quad \Longleftrightarrow \quad \mathrm{Halt}(\mathrm{CM}_{215}, (0, h, T)).$$

Proof. Apply positive-stuttering transfer to the three-clause invariant and the local simulation. Initial control 0 satisfies the invariant independently of T and h. The state map preserves and reflects the designated halt. The target terminal is absorbing in the totalized semantics. □

The new Lean theorem is `CM215.halts_iff`; its axiom report is `[propext, Quot.sound]`. The exact table generator checks the parent and alternative native byte hashes before emitting the common dense numbering. The 216th element of the semantic state type is the already existing halt; there are still 215 working states.

The bridge to the repository's actual `BusyBeaver.Core` semantics is also kernel-checked. The theorem `CM215.repo_blank_halts_iff` connects the exact `KDense215.machine215` definition to `machineFrom M`; all 430 parent transition rows are checked in the kernel. The parameterized `inherit_calibration` theorem then transfers any established halting characterization of that exact parent. The new generic lemma, concrete simulation, Core adapter, and repository bridge total 363 source lines including comments; the generated literal table data is a separate 462 lines. The clean six-module check took 27.543 seconds here, excluding the existing `KDense215` dependency build. These timings are a reproduction receipt, not a controlled performance comparison with the full primary proof.

At the mathematical level, composition with the repository's primary endpoint immediately gives

$$\mathrm{ZFC} \vdash (\neg\, \mathrm{Halt}(\mathrm{CM}_{215}, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{ZFC})).$$

For a fully compiled named endpoint in the repository, use the explicit machine-semantics adapter described in the artifact status, and compose with `KEndpoint215.blank_halts_iff_zfc_refutation`. This last composition reuses the entire existing calibration proof.

5 Which 215-state alternative should be formalized next?

The relevant comparison is the proof boundary that changes, not just the length of the source program or the number of edited transition rows. The repository's alternatives index distinguishes several such boundaries [7].

Alternative	Principal change	Consequence for proof effort
Conditional-marker CM215	Six local rows, unchanged source	Best immediate relative proof: three tape facts and bounded macros; no new source or startup argument.
Rebalanced addresses	Shallower source/address layout	May reduce decoder depth, but needs a new exact source/site/runtime correspondence. Shallower data alone does not establish shorter proofs.

Alternative	Principal change	Consequence for proof effort
Reencoded, additive-counter $D = 1$ start	Addresses and initial seed $N = 4, D = 1$	Requires a changed-seed lemma and exact physical allocation/startup correspondence; the enumeration cutoff remains 4.
Zero-spare variants	Increment refill and blank seed	Reuse the arbitrary-spare runtime contract, while checking append-zero refill, changed startup, and protected-tail credit.
Direct-transfer and MP-first transfer	Command operation and enumeration	Shorter command bodies may save implementation cost, but require the corresponding stack-credit, guard, and enumeration arguments.
Rho6 parameter variants	Sampler reparameterization	Keep the same numerical axioms; require the repaired axiom-selection compiler/runtime correspondence.

This is a ranking for obtaining a small, auditable additional correctness proof now. It is not a claim that CM215 would have the simplest proof if every machine had to be developed from scratch. Nor is it a claim that other variants cannot eventually produce a smaller machine.

For future alternatives, a practical interface is a certificate consisting of a configuration map, a small inductive invariant, terminal compatibility, and positive macro simulations. The invariant must describe configurations at simulation boundaries, not arbitrary controls inside a helper detour. That distinction prevents an overstrong and sometimes false all-control equivalence statement.

6 The attempt to reach 214 states

No verified 214-state machine was obtained in this investigation. We did implement and execute a larger local-fold search whose acceptance condition is deliberately chosen to support a short correctness transfer. The search is an additional reproducible negative result, not a lower bound.

The six inputs are literal native tables from the pinned ProveIt snapshot: primary K215, CM215 (conditional markers), ST215 (direct transfer), the zero-spare-transfer alternative, the MP-first-transfer alternative, and K215-R41 (rebalanced addresses). The input manifest records the complete repository path, Git blob identity, byte length, and SHA-256 digest of each table. Five declare initial control 0; zero-spare-transfer declares initial control 74. Every input has 215 working states, 430 rows, and separate halt label 254.

6.1 A finite invariant that does not depend on bounded blank runs

Fix a deterministic binary machine M and a radius $k \geq 1$. Its abstract states are

$$(q, v), \qquad q \in Q \cup \{H\}, \quad v \in \{0,1\}^{2k+1}.$$

Here v describes the cells at offsets $-k, \ldots, k$ from the current head. Start with *all* words v at the declared initial control. For each abstract transition, write the prescribed center bit, move the

window one cell, and allow either value for the newly exposed boundary bit. Let A_k be the least set closed under these transitions.

Window invariant. For every initial binary tape, every finite configuration reached from the declared initial control has its control/window pair in A_k.

Proof. The initial pair belongs by construction. Suppose the claim holds before one machine step. The actual read bit is the center of the stored window, so the abstract and actual transitions choose the same row and write the same bit. Their window shifts agree on every cell retained in the window. The one newly exposed actual bit is either zero or one, and both choices were admitted by the abstract rule. Closure of A_k proves the induction step. No bound on the number of actual steps or on the support of the initial tape is used. □

The implementation checks closure a second way: the fixed-point builder uses integer bit operations, whereas the control checker converts each word to a tuple and implements shifting by slicing and concatenation. This independent finite check is useful evidence about the implementation; it is not described as a proof-assistant kernel check.

6.2 A certificate format that transfers the existing endpoint

Choose distinct working controls a, b, retain a, and remove b. If the initial control is one of these, retain that control. Define

$$\phi(b) = a, \qquad \phi(q) = q \quad (q \neq b), \qquad \Phi(q, T, h) = (\phi(q), T, h).$$

Redirect every remaining transition target through ϕ. The two rows of the retained state may both be replaced; other rows are unchanged apart from target redirection. The following local obligation is enough: for each old entry $(q, v) \in A_k$ with $q \in \{a, b\}$, the proposed machine, starting in control a with window v, executes a positive finite block to the projection of the old one-step result. The block must have the same final head displacement and the same complete tape effect. It may not read outside the represented window.

Local-fold transfer theorem. If every local obligation holds, then the proposed machine N and M halt on exactly the same initial binary tapes, from their declared initial controls. Consequently an already proved blank-tape consistency equivalence for M transfers to N without re-proving the proof-search compiler or the calculus-to-ZFC reduction.

Proof. For an old step outside $\{a, b\}$, its projected counterpart is one new step. For a step at either merged control, the window invariant supplies an admitted local context, and the certificate supplies a positive block. Cells outside the window are untouched, so the local tape comparison is a comparison of the entire tapes. Induction on the length of an old run therefore constructs its projected new run. Old finite halting gives new finite halting, because ϕ preserves the distinct halt control. If the old run is infinite, the sum of positive block lengths is unbounded. The new deterministic run consequently has arbitrarily long nonhalting prefixes and cannot halt. These two cases give the equivalence. □

This is a conventional simulation argument; the contribution here is its use as the explicit synthesis contract and the reproducible search on the current tables. It is not presented as a new general theorem of automata theory.

6.3 Search family and measured result

The executed search used $k = 4$, hence nine-cell windows, and at most 48 new steps per old step. It examined all $\binom{215}{2} = 23{,}005$ state pairs for each table. It allows both retained rows to change, rather than requiring an identical row on the other read symbol. For each row, all choices of write bit, L/R move, and remaining working or halt destination are considered: $2 \cdot 2 \cdot 215 = 860$ actions.

The helper fragment must remain unchanged, and neither merged control may be visited internally before the matching endpoint. This restriction makes the two branch problems independent: the search indexes possible finite helper blocks, then intersects their admissible first actions over every abstract entry window. It need not enumerate the Cartesian product of two row-action sets. A helper is rejected if it exceeds the step bound, exits the window, halts early, or fails to reach a required matching endpoint within the bounds. Only the selected final boundary must match; intermediate visits to other possible endpoints are permitted.

Different old controls may admit the same abstract window and require different projected endpoints. This is *not* grounds for rejecting the pair: a common deterministic new trajectory can reach those endpoints at different positive times. The executed search permits this. Its index retains every distinct endpoint encountered along a helper trajectory, rather than committing to one global stopping time. An explicit control checks a common trajectory that matches one old branch in one step and the other in three steps.

Input	Abstract pairs	Branch searches	All folds	Survivors
Primary K215	27,579	23,187	23,005	0
CM215	27,723	23,180	23,005	0
ST215	27,783	23,182	23,005	0
Zero-spare transfer	27,957	23,188	23,005	0
MP-first transfer	27,773	23,187	23,005	0
K215-R41	27,615	23,190	23,005	0

"Abstract pairs" counts (q, v) in A_4, including admitted halt windows. "Branch searches" counts independent row-synthesis problems; the second read symbol is considered only if the first has a surviving action. Across the six tables, 138,030 state pairs were considered; the number is not a count of physically distinct candidate machines. The known 69/400 marker fold is recovered after explicitly unfolding it, with exactly three steps in all 16 admitted radius-three zero-branch contexts. An early-halt replacement and a wrong-selector replacement are both rejected. The direct local simulator and indexed block implementation agree on the positive control.

These results rule out neither longer blocks nor helpers that revisit a merged control, wider invariants, multiple simultaneous merges, changes to other rows, altered source programs, or different formalisms. Moreover the abstract invariant can admit contexts that no actual computation reaches. The result therefore says only that this bounded, explicitly stated proof method found no successor among these six inputs. The Python checker is ordinary executable evidence; this packet does not assert a newly compiled 214-state correctness theorem.

6.4 The next useful extension

The most promising extension of this particular method is to synthesize two or more local edits together, permitting a helper to revisit a rewritten control under a strictly decreasing finite rank. This keeps a finite termination certificate while escaping the independent-branch restriction. The objective should include the size of the required invariant and transfer proof as well as the state count. For a promising result, minimize its invariant to a small collection of bit predicates before generating Lean or Rocq code. Importing tens of thousands of abstract contexts into the kernel would defeat the proof-simplification goal when a few inductive clauses suffice, as they do for CM215.

The supplied reproduction entry point checks exact inputs and controls by default; `python3 search214/reproduce.py -search` additionally replays all six searches and compares every stable receipt field. No solver, network access, or third-party Python package is required.

6.5 Why the obvious selector deletion is not free

The conditional-marker construction merges two logical roles but spends the released physical label on a selector. Its state count is therefore unchanged. A genuine reduction would have to implement that selector using an existing compatible state or a provably terminating existing macro, while preserving all its other uses. Similar-looking rows do not suffice: the candidate's transient controls, tape writes, and return positions must be correct in every reachable boundary context.

The already retained return-scanner deletion example is a useful warning about the scope of experiments. The published CM215 packet gives a concrete valid-frame failure of a tempting 214-state deletion at addresses ending in six zeros. That counterexample rules out the proposed runtime contract; it does not by itself prove that every possible blank-run calibration of that table is impossible [6]. Our searches preserve this distinction.

6.6 A better synthesis target

The proof results suggest a focused next search. Optimize the pair

$$(\text{working states},\ \text{new proof obligations})$$

lexicographically or as a Pareto frontier. Let a synthesizer propose a state fold and a small local invariant together. A checker should accept only after verifying that the invariant is inductive for every relevant row, that the helper runs have positive length, and that their complete tape effects match. Counterexamples should refine the invariant or reject the proposed fold, rather than being discarded because they were not seen on a bounded blank run.

There are two useful extensions beyond the current search family. First, allow two modified helper states and a phase bit encoded in an already present marker. Second, allow the simulation boundary to shift by one cell, with an explicit tape relation. Either may expose a fold that a same-head, same-tape-boundary search cannot express. Neither is claimed to have succeeded here. Both can reuse positive-stuttering transfer once their local contracts are proved.

At the source level, quotienting equal effects and arranging shared continuations can also reduce physical cost, but a smaller source DAG does not automatically yield fewer binary states. Allocation, startup, and context-sensitive engine rows remain part of the count. Any future 214 claim must provide the complete literal table, not only a smaller source expression.

7 Integration and reproducibility

7.1 Suggested integration order

1. Add the generic positive-stuttering lemmas as small independent modules. Their terminal-persistence and positive-duration assumptions should be visible in their public statements.
2. Add the exact CM215 relative proof, table binding, and machine adapter. Keep the primary K215 endpoint as the calibration authority. Expose the alternative endpoint by transitivity.
3. Add the normalized source checker alongside the old checker. Retain the exact old-to-new certificate map and the source pin as audit data.
4. Replace the primary source-certificate facade only after the complete new certificate and exact program/tree equalities pass the kernel. Compare axiom reports and endpoint statements; do not infer success from a generator exit code.
5. Benchmark time and peak memory in a serial, pinned build after integration. Obligation counts and source bytes are already exact, but performance comparisons require comparable resource settings.

The delivered archive contains the report source and PDF, the new Lean and Coq modules, exact machine inputs, deterministic generators/checkers, certificate data, scoped state-search code and results, provenance, and verification logs. The README gives commands relative to the extracted archive and the repository root. Generated compiler objects and downloaded toolchains are not part of the deliverable.

7.2 Trusted and untrusted components

The Python generators and searchers are untrusted producers. The relevant formal theorems are accepted only when the assistant kernel checks their proof terms. Finite Boolean checks use ordinary kernel reflection; no `sorry`, `Admitted`, or custom consistency axiom is used in the new checked modules. Hashes bind copied machine bytes to a pinned source; they are not substitutes for the semantic proof.

The local blanket rebuild attempt is not reported as a successful fresh K215 endpoint replay. It exposed missing package registration for a repository-local dependency and the memory-heavy source-data expansion. Those logs are kept separate from the successful new theorem checks. The existing published K215 result remains the cited baseline; the new relative theorem, generic normalized checker, and complete replacement source theorem have their own direct kernel checks.

8 Further research questions

1. Can path normalization and shared tree reification become a common Lean/Rocq certificate format, so both kernels consume one source-specific data object with small independent checkers?
2. Can the CM215 configuration relation be expressed directly in the existing zipper semantics, eliminating one representation adapter and making subsequent local patches almost entirely finite certificates?

3. Can an existing runtime role implement the conditional selector under a stronger, still cheaply inductive boundary invariant, thereby turning the state-neutral marker fold into a genuine 214-state result?
4. How much additional compression is obtained by quotienting source checkpoints by register-effect bisimulation after removing trailing-zero duplication? What is the extra soundness proof cost?
5. Can proof-aware synthesis discover its own finite-state invariant and positive ranking function, rather than relying on a chosen window radius and macro bound?
6. Which remaining part of the external K215 equivalence is the most economical bridge to an internal PA proof, and which of the new finite certificates can be reused verbatim there?

The immediate mathematical result is a small checked route from an existing 215-state endpoint to a distinct 215-state machine. The immediate proof-engineering result is an exact reduction in source-certificate redundancy with a checked semantic justification. A 214-state construction remains open after the searches reported here.

References

[1] ProveIt, *KEndpoint215.lean*, pinned commit `b6c7087477`.
Exact Lean endpoint and its component interfaces.

[2] ProveIt, *k225-formal/README.md*, same pin.
Formal proof architecture and the 215-state construction.

[3] ProveIt, *verification/k215/lean/README.md*, same pin.
Historical fresh verification, current closure, and reproduction boundary.

[4] ProveIt, *simplification/phase3/README.md*, same pin.
Already adopted simplifications and their checked status.

[5] ProveIt, commits `1d264e552f` and `b6c7087477`.
Merged symbolic-execution and step-soundness refactors.

[6] ProveIt, *candidate-k215/alternatives/conditional-marker*, same pin.
Published CM215 table, ordinary proof, attribution, and negative examples.

[7] ProveIt, *candidate-k215/alternatives/README.md*, same pin.
Index of materially distinct 215-state alternatives.